

Inventory MCP Project — Architecture Overview

How a request travels from the React frontend, through the Express/SQLite backend, to the MCP server and Claude — for both plain data pages and the AI Assistant.

1. The Three Processes

Frontend — React + Vite

project2/frontend · runs on `http://localhost:5173` · pages call functions in `src/api/*.js`, which all funnel through `src/api/client.js` (fetch wrapper)

HTTP (fetch), JSON over REST

Backend — Node/Express + SQLite

project2/backend · runs on `http://localhost:4001` · routes → controllers → services → SQLite (`inventory.db`) · also hosts `POST /api/ai/chat`

stdio (child process), spawned by the backend

MCP server — 25 tools

project2/mcp-server · no port — communicates only over stdin/stdout · each tool proxies back to the same backend REST API over HTTP

Only **two** of these are long-running network servers (frontend dev server, backend API). The MCP server has no server socket at all — it is a short-lived child process the backend spawns on demand and talks to over stdin/stdout using the Model Context Protocol.

2. Flow A — A Normal Page (e.g. Products)

This is the path every page except AI Assistant uses. No AI, no MCP — straight REST.

- 1 `Products.jsx` mounts → calls `productsApi.getProducts()` from `src/api/products.js`
- 2 `products.js` calls `get("/products")`, defined in `src/api/client.js` — a thin `fetch()` wrapper that prefixes `VITE_API_BASE` (default `http://localhost:4001/api`)
- 3 Express receives `GET /api/products` → `routes/productRoutes.js` → `controllers/productController.js#listProducts`
- 4 Controller calls `services/productService.js`, which runs SQL against `db/inventory.db` (SQLite, via Node's built-in `node:sqlite`)
- 5 JSON flows back up: service → controller → Express response → `fetch()` in the browser → `setProducts()` → table renders

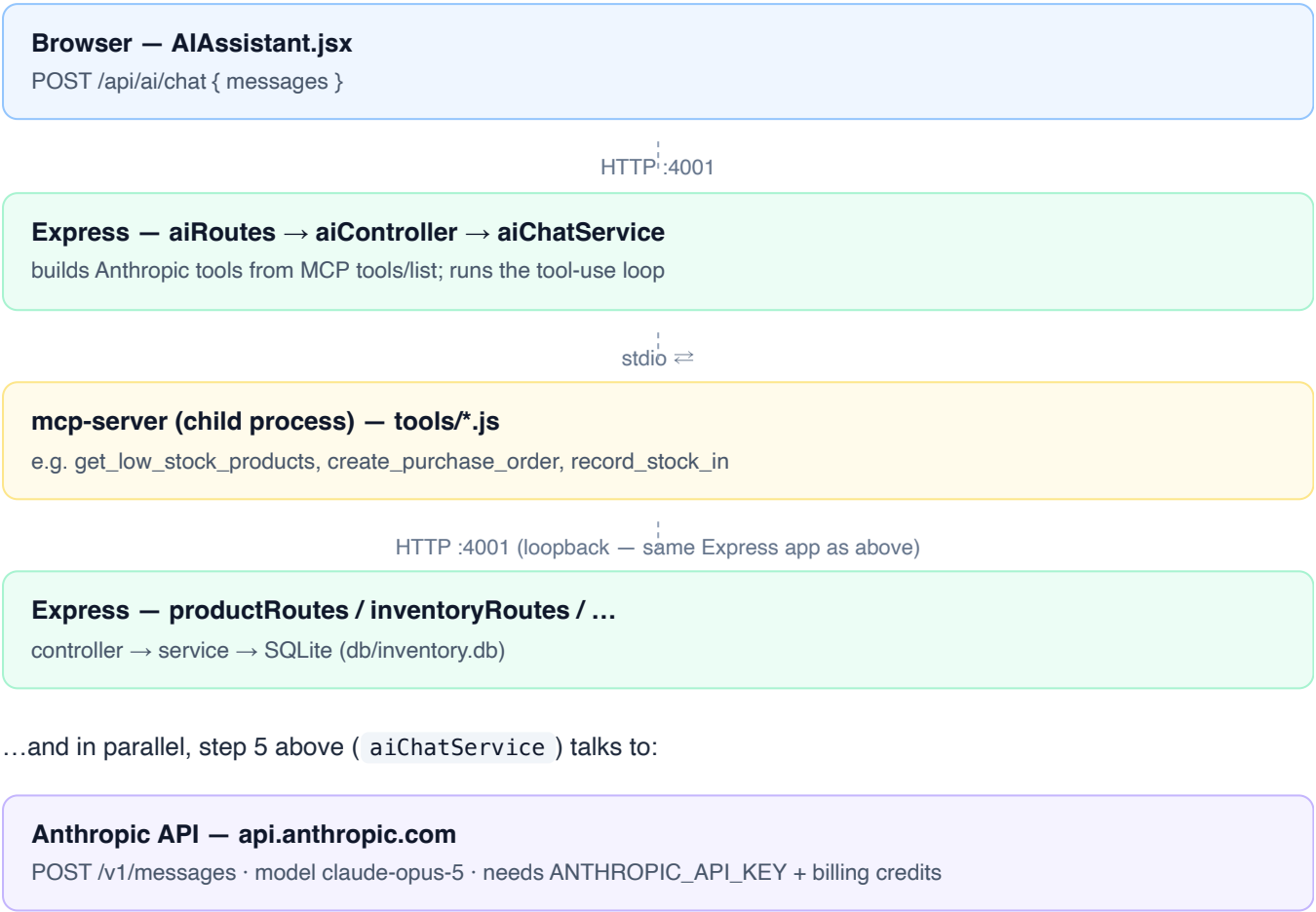
3. Flow B — The AI Assistant (the MCP-backed path)

This is the interesting one — it involves all three processes and an external call to Anthropic's API.

- 1 **Frontend:** user types a question in `AIAssistant.jsx` → `aiApi.sendMessage(history)` (`src/api/ai.js`) → `POST /api/ai/chat` with `{ messages: [{role, text}, ...] }`
- 2 **Backend routing:** `routes/aiRoutes.js` → `controllers/aiController.js#chat` validates the body and calls `services/aiChatService.js#chat(messages)`
- 3 **Connect to MCP (first call only):** `services/mcpClient.js` spawns `mcp-server/server.js` as a child process (`StdioClientTransport + MCP SDK Client`), then caches that connection (a module-level promise) so every later chat request reuses the same subprocess instead of respawning it
- 4 **Discover tools:** `aiChatService` calls `listMcpTools()` → `MCP tools/list` → the 25 registered tools (`get_products`, `create_purchase_order`, `record_stock_in`, ...) come back with their JSON-Schema `inputSchema`, which is reshaped into Anthropic's `{name, description, input_schema}` tool format
- 5 **Call Claude:** `client.messages.create({ model: "claude-opus-5", system, tools, messages })` via `@anthropic-ai/sdk`, sent to Anthropic's API over the internet
- 6 **Tool-use loop:** if `stop_reason === "tool_use"`, the assistant's turn (containing one or more `tool_use` blocks) is appended to the message list, then for each block:
 - `callMcpTool(name, input)` → `MCP tools/call` over stdio → the MCP server's tool handler (e.g. `tools/productTools.js`) runs
 - That handler calls the **same backend REST API** over plain HTTP (`mcp-server/services/apiClient.js` → `GET/POST http://localhost:4001/api/...`) — i.e. the MCP server is itself a REST client of the Express app, just like the frontend is
 - The result is wrapped as a `tool_result` block and the whole batch is sent back to Claude in the next `messages.create` call
- 7 This repeats (capped at 6 iterations) until Claude replies with plain text (`stop_reason !== "tool_use"`) — that text is extracted and returned as `{ reply }`
- 8 **Frontend:** the reply is appended to the chat as an assistant message and rendered

Why spawn the MCP server instead of calling the services directly? It reuses the exact tool definitions, descriptions and input schemas already written and tested in Phase 3 (`mcp-server/tools/*.js`) — the same MCP server that Claude Code itself talks to via `.mcp.json` — instead of re-declaring 25 tool schemas a second time in the backend.

4. End-to-End Diagram — AI Assistant Request



5. Key Files

Layer	File	Role
Frontend	src/api/client.js	fetch() wrapper — the only place fetch lives
Frontend	src/api/ai.js	sendMessage(messages) → POST /ai/chat
Frontend	src/pages/AIAssistant.jsx	chat UI, message history, loading/error state
Backend	routes/aiRoutes.js	POST /chat route
Backend	controllers/aiController.js	request validation, error → 500 JSON
Backend	services/aiChatService.js	the Claude tool-use loop itself
Backend	services/mcpClient.js	spawns & caches the MCP stdio connection
MCP server	server.js	registers all 25 tools, connects StdioServerTransport
MCP server	tools/*.js	one file per resource (products, suppliers, purchase orders, inventory)
MCP server	services/apiClient.js	fetch wrapper — proxies every tool call to the backend REST API

6. Configuration & Requirements

Variable	Where	Purpose
VITE_API_BASE	frontend/.env	Backend URL the frontend calls (default http://localhost:4001/api)
PORT	backend/.env	Port the Express API listens on (default 4001)
ANTHROPIC_API_KEY	backend/.env	Required for POST /api/ai/chat — without it, the endpoint returns a clear error instead of crashing
ANTHROPIC_MODEL	backend/.env	Optional override, defaults to claude-opus-5
API_BASE	passed to the mcp-server child process	Tells the MCP server's tools which backend URL to call — set automatically by mcpClient.js

Both the backend and the MCP server must ultimately reach the same SQLite-backed REST API. Whether that request comes from a browser page, or from Claude via an MCP tool call, it lands on the exact same Express routes — there is only one source of truth for the data.